

Reporte: Typi

Luana Sofia Alvarez Molina

17 de junio de 2026

Índice

1. Introducción	2
2. Análisis	2
2.1. Objetivos	2
3. Diseño	3
3.1. Arquitectura del servidor	3
3.2. Arquitectura del cliente	3
3.3. Mecanismos de Concurrencia y Sincronización:	3
3.4. Patrones de Diseño:	4
3.5. Diagrama de clases	4
4. Implementación	5
4.1. Modulo del Servidor	5
4.1.1. Red e Inicialización:	5
4.1.2. Entidades	7
4.1.3. Gestores:	9
4.2. Módulo del cliente	10
5. Dificultades	10
6. Resultados	11
7. Conclusiones	11

Resumen

Desarrollo de un sistema de chat en tiempo real tipo IRC mediante una arquitectura cliente-servidor. Incluye la gestión de conexiones simultáneas mediante hilos de ejecución (multithreading) y persistencia de estados, aplicando programación concurrente, manejo de sockets y un diseño modular para la comunicación síncrona. Aunque hubo limitaciones de tiempo, la parte del servidor se completó con éxito, dejando una infraestructura funcional que sirve como base sólida para el desarrollo futuro de la parte del cliente.

1. Introducción

El objetivo de este proyecto es desarrollar un sistema de chat en tiempo real tipo IRC que permita el intercambio síncrono de mensajes y la gestión de salas de conversación siguiendo el modelo cliente-servidor.

El sistema está compuesto por dos módulos principales: el servidor, encargado de abrir los puertos de comunicación, escuchar conexiones entrantes, gestionar de forma concurrente a los usuarios mediante hilos de ejecución y validar las reglas del protocolo; y el cliente, diseñado para ejecutarse en la terminal y servir como la interfaz que permite al usuario interactuar, enviar peticiones y recibir los mensajes en tiempo real.

Este proyecto busca integrar conceptos de programación concurrente (multihilos), comunicación en red mediante sockets TCP/IP, y el modelo cliente-servidor. Como parte de los alcances de esta entrega, el desarrollo se centró en consolidar un backend robusto en Go tras una migración estratégica desde el lenguaje C. A través de este proceso, se lograron implementar mecanismos estrictos de defensa en el servidor para validar las estructuras JSON entrantes mediante la segmentación por bytes nulos (`\0`). A pesar de enfrentar retos técnicos debido a la falta de experiencia previa en computación concurrente y contratiempos con la gestión del tiempo para el cliente, los resultados obtenidos validan la estabilidad del servidor mediante pruebas de emulación de red, dejando el sistema listo para la integración del módulo del cliente así como su respectiva interfaz gráfica.

2. Análisis

2.1. Objetivos

- Desarrollar un sistema de chat multiusuario en tiempo real usando el protocolo proporcionado bajo una arquitectura cliente-servidor robusta y eficiente
- Implementar un mecanismo de concurrencia en el servidor que permita atender y procesar peticiones de múltiples clientes de forma simultánea sin que colapse.
- Establecer canales de comunicación mediante el uso de sockets.
- Validar de manera estricta las restricciones del protocolo, incluyendo el límite de caracteres para identificadores (8 para usuarios y 16 para salas), la unicidad de nombres y la gestión de los estados disponibles (ACTIVE, AWAY, BUSY).
- Permitir la creación y administración dinámica de cuartos de conversación virtuales, controlando los flujos de invitación, unión y abandono de cuartos, así como la destrucción automática de estos en caso de estar vacíos.
- Garantizar la correcta difusión de mensajes, soportando tanto la comunicación privada como el envío global a canales públicos o cuartos específicos.
- Diseñar un cliente interactivo por terminal que funcione como interfaz de usuario para enviar los mensajes y recibir mensajes.
- Implementar un sistema de control de excepciones que desconecte automáticamente a aquellos clientes que envíen mensajes con formatos inválidos, incompletos o sin haberse identificado previamente.

- (Opcional) Implementar una interfaz gráfica para el cliente.

3. Diseño

El sistema se diseñó bajo una arquitectura modular distribuida dividida en dos grandes componentes independientes: el servidor y el cliente. El flujo de comunicación se basa en el intercambio de mensajes en formato JSON a través de sockets TCP, donde el servidor actúa como el nodo central que coordina y distribuye la información.

3.1. Arquitectura del servidor

Para mantener un orden claro y separar las responsabilidades, el servidor se dividió en tres módulos principales:

Los módulos del sistema son:

- **Módulo de Red e Inicialización:** Es el punto de entrada del programa. Se encarga de capturar el puerto indicado por el usuario, levantar el socket TCP principal y quedarse escuchando de forma indefinida en un ciclo. Cada vez que detecta que un cliente intenta conectarse, acepta la conexión y delega su atención inmediatamente a una subrutina concurrente para no bloquear el sistema.
- **Módulo de Entidades:** Define las estructuras de datos fundamentales para modelar el estado del chat:
 - **Cliente:** Representa la abstracción de un usuario conectado, almacenando su socket de red, su nombre, su estado actual (ACTIVE, AWAY, BUSY) y su hilo de ejecución correspondiente.
 - **Cuarto:** Representa una sala virtual. Utiliza diccionarios para gestionar de manera eficiente tanto a los usuarios participantes como a la lista de usuarios invitados que tienen permiso de unirse
- **Módulo de Gestores:** Contiene la lógica de negocio y las reglas del protocolo. Analiza los JSON recibidos por el cliente y manda llamar a la función correspondiente para validar las restricciones (como que el nombre de usuario no pase de 8 caracteres o el cuarto de 16). También maneja las respuestas de éxito, de error (como intentar unirse a un cuarto sin invitación) y las funciones auxiliares para difundir los mensajes a todos los usuarios o a cuartos específicos.

3.2. Arquitectura del cliente

El cliente se diseñó como una aplicación de terminal dividida en dos subcomponentes que trabajan al mismo tiempo:

- **Interfaz de Usuario:** Encargada de leer continuamente lo que el usuario escribe en el teclado, estructurar la petición en el formato JSON correspondiente al protocolo y enviarla a través del socket hacia el servidor.
- **Escucha de red:** Encargada de quedarse esperando y leyendo los datos que llegan desde el servidor. Al recibir un JSON, lo decodifica y pinta el mensaje en la pantalla del usuario en tiempo real sin interrumpir la escritura en la terminal.

3.3. Mecanismos de Concurrencia y Sincronización:

Dado que el chat debe soportar múltiples usuarios interactuando de forma simultánea, el diseño integra herramientas específicas para evitar conflictos al acceder a la información compartida (como la lista global de usuarios conectados o los participantes de una sala).

- **Subrutinas Concurrentes:** El servidor asigna una rutina independiente para la lectura y procesamiento de cada cliente. Esto permite que las operaciones de red de un usuario no congelen la aplicación para los demás.
- **Exclusión Mutua:** Para evitar problemas al momento de modificar información compartida en el servidor (por ejemplo, que dos rutinas intenten borrar o agregar un usuario al mismo diccionario al mismo tiempo), tanto la estructura del servidor, de los cuartos y de cada cliente individual cuentan con seguros de exclusión mutua (sync.Mutex). Estos aseguran que solo una rutina pueda modificar o leer los datos compartidos a la vez, garantizando la consistencia del sistema.

3.4. Patrones de Diseño:

Durante el desarrollo del sistema se emplearon dos patrones de diseño de manera implícita, con el objetivo de mejorar la organización, mantenibilidad y claridad del código.

Por un lado, se implemento implícitamente el patrón de *Modelo-Vista-Controlador (MVC)*, pues aunque es un sistema ejecutado directamente en terminal, el servidor separa de forma clara sus componentes bajo este esquema. El Modelo está representado por las estructuras de datos (Cliente, Cuarto y las definiciones del protocolo); la Vista corresponde a la terminal interactiva de los clientes que pintan los mensajes; y el Controlador es todo el conjunto de gestores (gestorUsuarios.go, gestorCuartos.go, gestorChats.go) que reciben las peticiones de red, validan las reglas del juego y modifican el estado de los modelos.

Por otro lado, se uso explícitamente el patrón *Mediador* ya que el servidor central funciona como un mediador directo entre todos los usuarios. Los clientes están completamente aislados entre sí y no conocen la dirección de red de los demás; cuando un usuario quiere comunicarse con otro o con una sala, le delega esa responsabilidad al servidor mediante un JSON, y este se encarga de buscar al destinatario y despachar el mensaje.

3.5. Diagrama de clases

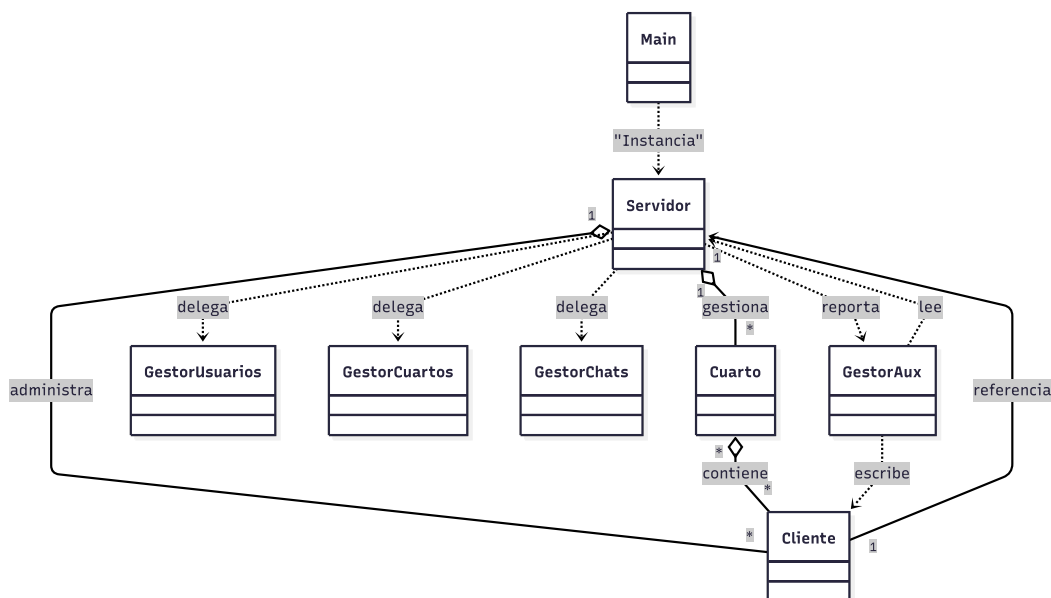


Figura 1: Diagrama de clases simplificado.

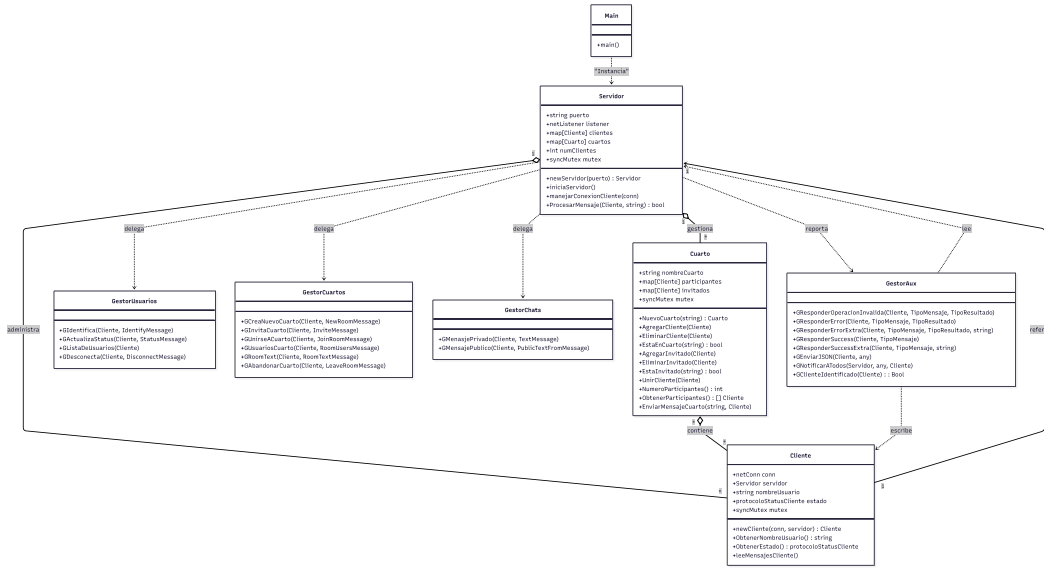


Figura 2: Diagrama de clases completo.

umlCompleto.pdf [Haga clic aquí para extraer el diagrama completo \(PDF\)](#)

4. Implementación

El sistema fue implementado utilizando el paradigma de programación orientada a objetos, empleando el lenguaje Go para el servidor debido a su no tan alta curva de aprendizaje, su óptimo rendimiento en red y su facilidad para manejar concurrencia nativa a través de goroutines, lo que permitió cumplir con los requisitos del protocolo sin la necesidad de usar bibliotecas externas.

La implementación se realizó siguiendo la arquitectura modular y los patrones de diseño mencionados previamente.

Adicionalmente, se optó por utilizar nombres de estructuras y funciones en español (como Cliente, Cuarto y gestorUsuarios), combinados con términos del protocolo en inglés, con el fin de facilitar la comprensión de la lógica de negocio y mantener una correspondencia directa con las instrucciones JSON solicitadas. Esto permite identificar rápidamente qué componentes pertenecen al diseño propio del servidor y cuáles corresponden estrictamente a la especificación del protocolo externo.

De la misma manera, se adoptó el uso de la convención **CamelCase** (tanto **upperCamelCase** para tipos de datos como **lowerCamelCase** para variables y métodos locales), en estricta concordancia con el estilo nativo y las guías de diseño del lenguaje Go.

4.1. Modulo del Servidor

El servidor se implementó bajo la estructura modular antes descrita, orientada a objetos y dividida en capas lógicas de responsabilidad, distribuidas en varios archivos independientes.

4.1.1. Red e Inicialización:

Este submódulo está comprendido por los archivos **main.go** y **servidor.go**. El archivo **main.go** funciona como el punto de arranque que lee el puerto desde los argumentos de la terminal e instancia la estructura principal. Por otro lado, **servidor.go** centraliza el núcleo de la comunicación distribuida. Su responsabilidad es abrir el socket TCP principal y mantener un ciclo infinito aceptando conexiones mediante **listener.Accept()**.

Para evitar que el servidor se congele atendiendo a un solo usuario, cada vez que entra un cliente se delega su atención inmediatamente a una subrutina concurrente (goroutine) mediante la instrucción `go s.manejarConexion(conexion)`. Además, este submódulo actúa como un despachador central (router), recibe las cadenas JSON crudas, identifica el campo "type" y realiza la decodificación hacia el struct de datos correspondiente antes de pasarlo a la capa de lógica.

Código: Inicialización del servidor

```
func main() {
    if len(os.Args) != 2 {
        fmt.Println("Uso:")
        fmt.Println("go run main.go <puerto>")
        return
    }
    var puerto string = os.Args[1]
    var servidor *Servidor = newServidor(puerto)
    servidor.iniciaServidor()
}
```

Código: Controlador de mensajes del protocolo (Router)

El método central (el corazón) de `servidor.go` encargado de validar las instrucciones todas las instrucciones del protocolo y redirigir los datos al gestor correspondiente mediante un flujo condicional de tipo `switch`.

```
func (s *Servidor) ProcesarMensaje(cliente *Cliente, mensaje string) bool {

    var mensajeJSON []byte = []byte(mensaje)
    var mensajeBase protocolo.MensajeBase
    var err error
    err = json.Unmarshal(mensajeJSON, &mensajeBase)
    if err != nil {
        fmt.Println("JSON inválido:", err)
        GResponderOperacionInvalida(cliente,
                                    protocolo.Invalid,
                                    protocolo.ResultadoInvalido)

        return false
    }

    switch mensajeBase.Type {
    case protocolo.Identify:
        var msj protocolo.IdentifyMessage
        var err error = json.Unmarshal(mensajeJSON, &msj)
        if err != nil {
            GResponderOperacionInvalida(cliente,
                                        protocolo.Invalid,
                                        protocolo.ResultadoInvalido)

            return false
        }
        GIdentifica(cliente, &msj)

    case protocolo.Status:
        var msj protocolo.StatusMessage
        var err error = json.Unmarshal(mensajeJSON, &msj)
        if err != nil {
```

```

        GResponderOperacionInvalida(cliente ,
                                    protocolo.Invalid ,
                                    protocolo.ResultadoInvalido)

        return false
    }
    GActualizaStatus(cliente , &msj)

case protocolo.Text:
    var msj protocolo.TextMessage
    var err error = json.Unmarshal(mensajeJSON, &msj)
    if err != nil {
        GResponderOperacionInvalida(cliente ,
                                    protocolo.Invalid ,
                                    protocolo.ResultadoInvalido)

        return false
    }
    GMensajePrivado(cliente , &msj)

    ...
    ...
    ...

case protocolo.RoomText:
    var msj protocolo.RoomTextMessage
    var err error = json.Unmarshal(mensajeJSON, &msj)
    if err != nil {
        GResponderOperacionInvalida(cliente ,
                                    protocolo.Invalid ,
                                    protocolo.ResultadoInvalido)

        return false
    }
    GRoomText(cliente , &msj)

default:
    GResponderOperacionInvalida(cliente ,
                                protocolo.Invalid ,
                                protocolo.ResultadoInvalido)

    return false

}

return true
}

```

4.1.2. Entidades

Este submódulo se encarga de modelar y mantener el estado de los objetos del chat dentro del servidor.

En `cliente.go` se abstrae a un usuario conectado, guarda su descriptor de archivo (`net.Conn`), su nombre de usuario único y su estado actual (ACTIVE, AWAY, BUSY). Contiene el ciclo encargado de escuchar activamente los bytes del socket de ese cliente específico.

En `cuarto.go` se modelan los cuartos virtuales de conversación. Almacena el nombre del cuarto y gestiona dos

listas clave: los participantes activos y los usuarios que han sido invitados. Para garantizar la eficiencia, estas listas se diseñaron utilizando diccionarios (`map[string]`), lo que permite validar invitaciones o verificar la existencia de un usuario en tiempo constante $O(1)$. Toda la estructura está protegida por un seguro de exclusión mutua (`sync.Mutex`) para evitar que dos hilos alteren las listas simultáneamente.

Código: Procesamiento del flujo de red y reconstrucción de mensajes

El corazón de `cliente.go` encargado de escuchar de forma indefinida el socket del usuario, concatenar los bytes entrantes en un búfer acumulador y segmentar las cadenas JSON únicamente cuando se detecta el caracter de terminación nulo (`\x00`).

```
func (c *Cliente) leeMensajesCliente() {

    defer func() {
        fmt.Println("Limpiando socket")
        msg := protocolo.DisconnectMessage{
            Type: protocolo.Disconnect,
        }
        GDesconecta(c, &msg)
    }()

    buffer := make([]byte, 1024)
    acumulado := ""

    for {
        n, err := c.conn.Read(buffer)
        if err != nil {
            if err == io.EOF {
                fmt.Println("Cliente cerró la conexión")
            } else {
                fmt.Println("Error de lectura:", err)
            }
            return
        }

        // Agregamos lo leído al acumulador
        acumulado += string(buffer[:n])

        if strings.Contains(acumulado, "\n") && !strings.Contains(
            acumulado, "\x00") {
            fmt.Println("Cliente inválido:
            ----- usa \\n en lugar de \\0. Desconectando ...")
            return
        }

        // Procesamos todos los mensajes completos (\0)
        for strings.Contains(acumulado, "\x00") {

            partes := strings.SplitN(acumulado, "\x00", 2)

            mensaje := partes[0]
            acumulado = partes[1]

            fmt.Println("Procesando:", mensaje)
```



```

        continua := c.servidor.ProcesarMensaje(c, mensaje)
        if !continua {
            nombre := c.ObtenerNombreUsuario()
            if nombre == "" {
                fmt.Println("Deteniendo lectura de mensajes para cliente no identificado.")
            } else {
                fmt.Printf("Deteniendo lectura de mensajes para %s por error de protocolo.\n", nombre)
            }
            return
        }
    }
}

```

4.1.3. Gestores:

Contiene las reglas e invariantes del protocolo dadas por la especificación. Recibe las estructuras de datos ya decodificadas y ejecuta las validaciones y acciones del chat.

En `gestorUsuarios.go` se controla el ciclo de vida del usuario, procesando la identificación inicial (`IDENTIFY`), asegurando que el nombre no pase de 8 caracteres, que sea único en el servidor, y manejando la desconexión limpia (`DISCONNECT`) que remueve al usuario de todas las salas.

Luego, en `gestorCuartos.go` se validan los flujos de los cuartos virtuales (creación con límite de 16 caracteres, envío de invitaciones múltiples y uniones autorizadas). Así como el envío de mensajes e interacciones dentro de los cuartos.

En `gestorChats.go` se coordina el envío de mensajes separando si la comunicación es privada (`TEXT`) o global (`PUBLIC_TEXT`).

Por último, en `gestorAux.go` se centralizan las tareas repetitivas de infraestructura como empaquetar structs a bytes JSON, añadir el carácter nulo de terminación (`\0`) y escribir de vuelta en los canales de red de los clientes.

Código: Identificación de un cliente

El siguiente fragmento de código muestra cómo `gestorUsuarios.go` aplica estas validaciones de negocio al momento de identificar un cliente en el sistema.

```

func GIdentifica(cliente *Cliente, msg *protocolo.IdentifyMessage) {
    if msg.Username == "" || len(msg.Username) > 8 {
        GResponderOperacionInvalida(cliente, protocolo.Invalid,
            protocolo.ResultadoInvalido)
        return
    }

    if cliente.ObtenerNombreUsuario() != "" {
        GResponderOperacionInvalida(cliente, protocolo.Invalid,
            protocolo.ResultadoInvalido)
        return
    }
}

```

```

//map access multi-value return -> mapaccess2
cliente.servidor.mutex.Lock()
_, existe := cliente.servidor.clientes[msg.Username]
if existe {
    cliente.servidor.mutex.Unlock()
    GResponderError(cliente, protocolo.Identify, protocolo.
        UserAlreadyExists)
    return
}

cliente.nombreUsuario = msg.Username
cliente.estado = protocolo.Active
cliente.servidor.clientes[msg.Username] = cliente
cliente.servidor.mutex.Unlock()

GResponderSuccess(cliente, protocolo.Identify)

mensajeJSON := protocolo.NewUserMessage{
    Type:      "NEW_USER",
    Username:  cliente.ObtenerNombreUsuario(),
}

GNotificarATodos(cliente.servidor, mensajeJSON, cliente)
}

```

4.2. Módulo del cliente

El módulo del cliente no fue implementado por motivos de tiempo y organización, quedando como trabajo futuro.

5. Dificultades

A lo largo del desarrollo del proyecto se presentaron diversas dificultades tanto técnicas como de organización:

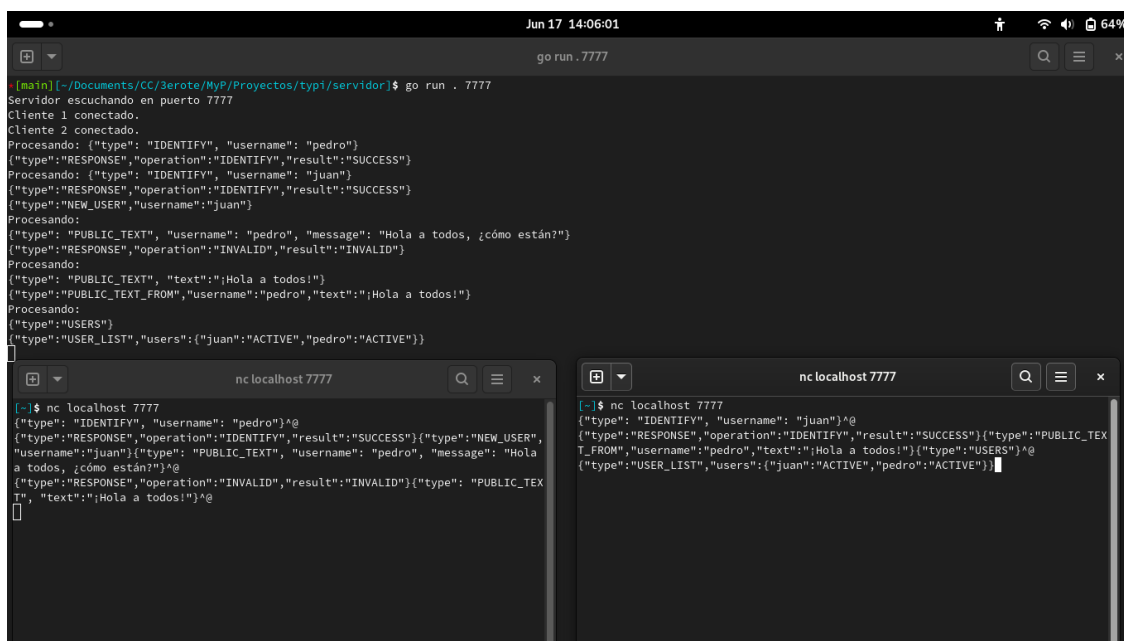
- **Lenguaje:** Se tuvieron dificultades iniciales con los lenguajes a usar, pues primero desarrollé la parte de inicialización del servidor en lenguaje C. Sin embargo, al ver lo complicado que se ponía el manejo manual de la memoria, decidí migrar el servidor a Go; esto ocasionó una pérdida de tiempo valioso en el cronograma.
- **Implementación del cliente:** Debido a la migración del servidor y al tiempo perdido en ello, el siguiente problema que enfrenté fue intentar desarrollar el cliente usando Rust. Esto implicó una doble curva de aprendizaje en paralelo; desafortunadamente, la barrera técnica de Rust era demasiado alta para el tiempo disponible, por lo que no fue posible dominarlo lo suficiente para construir dicho módulo.
- **Gestión del tiempo:** Como consecuencia directa de los dos puntos anteriores (la reescritura del servidor y el intento fallido con Rust), el tiempo restante se consumió por completo. Esto provocó un impacto crítico en el desarrollo general, obligándome a priorizar exclusivamente la estabilidad del servidor y dejando fuera por completo la implementación del cliente, el cual no se pudo desarrollar en lo absoluto, quedando como trabajo futuro junto con su interfaz gráfica.
- **Experiencia:** Al ser este mi primer proyecto grande de desarrollo y no tener tanto conocimiento de programación concurrente se me dificultó mucho el desarrollo del mismo. Pues tuve dudas en todo, desde la organización de directorios hasta la forma correcta de implementar. Es por esto que el hecho de no haber podido desarrollar el cliente, además del tiempo se lo atribuyo a mi nula experiencia desarrollando algo de este estilo y mi poco conocimiento de los conceptos que abordaban el proyecto.

6. Resultados

A pesar de las complicaciones temporales que impidieron el desarrollo de una aplicación cliente propia, se completó con éxito el núcleo del sistema. El servidor está completamente operativo, es robusto ante conexiones concurrentes y cumple estrictamente con el protocolo diseñado.

El servidor en Go se inicializa correctamente, escucha en el puerto asignado y es capaz de gestionar múltiples hilos de ejecución (*goroutines*) para atender a varios usuarios en paralelo sin presentar condiciones de carrera (*race conditions*).

Al no contar con un cliente, la verificación de la lógica de negocio se realizó emulando un cliente real a través de la herramienta de red `nc`. Las pruebas demostraron que el servidor responde con éxito (mensajes de `SUCCESS`) cuando los datos son correctos y activa sus mecanismos de defensa (mensajes de `INVALID` o desconexión inmediata) cuando se viola el protocolo, como al exceder los 8 caracteres en el nombre de usuario o al intentar usar saltos de línea (`\n`) en lugar del byte nulo (`\0`).



```
[main] [~/Documents/CC/3erote/MyR/Proyectos/typ1/servidor]$ go run . 7777
Servidor escuchando en puerto 7777
Cliente 1 conectado.
Cliente 2 conectado.
Procesando: {"type": "IDENTIFY", "username": "pedro"}
{"type": "RESPONSE", "operation": "IDENTIFY", "result": "SUCCESS"}
Procesando: {"type": "IDENTIFY", "username": "juan"}
{"type": "RESPONSE", "operation": "IDENTIFY", "result": "SUCCESS"}
{"type": "NEW_USER", "username": "juan"}
Procesando:
{"type": "PUBLIC_TEXT", "username": "pedro", "message": "Hola a todos, ¿cómo están?"}
{"type": "RESPONSE", "operation": "INVALID", "result": "INVALID"}
Procesando:
{"type": "PUBLIC_TEXT", "text": ";Hola a todos!"}
{"type": "PUBLIC_TEXT_FROM", "username": "pedro", "text": ";Hola a todos!"}
Procesando:
{"type": "USERS"}
{"type": "USER_LIST", "users": {"juan": "ACTIVE", "pedro": "ACTIVE"}}
[]

nc localhost 7777
[~]$ nc localhost 7777
{"type": "IDENTIFY", "username": "pedro"}^@
{"type": "RESPONSE", "operation": "IDENTIFY", "result": "SUCCESS"}{"type": "NEW_USER", "username": "juan"}{"type": "PUBLIC_TEXT", "username": "pedro", "message": "Hola a todos, ¿cómo están?"}^@
{"type": "RESPONSE", "operation": "INVALID", "result": "INVALID"}{"type": "PUBLIC_TEXT", "text": ";Hola a todos!"}^@

nc localhost 7777
[~]$ nc localhost 7777
{"type": "IDENTIFY", "username": "juan"}^@
{"type": "RESPONSE", "operation": "IDENTIFY", "result": "SUCCESS"}{"type": "PUBLIC_TEXT_FROM", "username": "pedro", "text": ";Hola a todos!"}{"type": "USERS"}^@
{"type": "USER_LIST", "users": {"juan": "ACTIVE", "pedro": "ACTIVE"}}[]
```

Figura 3: Validación de la lógica del servidor y respuesta al protocolo JSON utilizando `nc`.

7. Conclusiones

El desarrollo de este proyecto permitió poner en práctica conceptos avanzados de programación concurrente, diseño de protocolos de comunicación a nivel de red y gestión de proyectos de software.

La migración estratégica del servidor desde el lenguaje C hacia Go demostró ser un acierto arquitectónico. Go proporcionó las abstracciones nativas necesarias (*goroutines* y *mutex*) para gestionar de forma segura y eficiente la concurrencia de múltiples clientes, eliminando los riesgos de fugas de memoria y simplificando el control de hilos en comparación con la gestión manual de memoria en C.

La implementación de los componentes de validación (*gestores*) demostró que un protocolo de red debe ser estricto. La estrategia de segmentación mediante el byte nulo (`\0`) y el rechazo inmediato de caracteres de salto de línea (`\n`) probó ser un mecanismo de defensa efectivo para garantizar la integridad de las estructuras JSON entrantes y proteger al servidor de peticiones malformadas o clientes inválidos.

El proyecto me dejó una lección crítica respecto a la gestión del tiempo y la evaluación de riesgos tecnológicos. Intentar adoptar múltiples herramientas desconocidas en paralelo (Go para el servidor y Rust para el cliente) introdujo una curva de aprendizaje doble que redujo drásticamente el margen de tiempo de maniobra, imposibilitando el desarrollo del módulo del cliente.

Al ser este el primer proyecto de este estilo en la carrera y al no haber cursado aún la materia formal de Computación Concurrente, me enfrenté a una brecha considerable de conocimiento técnico. Esto incrementó la dificultad en aspectos fundamentales que abarcaron desde la estructuración idónea de directorios en Go hasta la lógica de sincronización de hilos. Sin embargo, a pesar de esta falta de experiencia previa, considero que el diseño del servidor se consolidó de forma exitosa y completamente funcional, validando la capacidad de asimilación de conceptos complejos bajo presión y premura del tiempo.

Dado que la parte del modelo o bien el backend quedó 100 % operativo y verificado mediante herramientas de diagnóstico de red como `nc` (Netcat), el proyecto establece bases sólidas para su continuidad, quedando así como trabajo futuro el desarrollo integral de la aplicación del cliente, así como el diseño de su correspondiente interfaz gráfica de usuario, permitiendo aprovechar el protocolo concurrente ya establecido en el servidor.

En conclusión, el sistema desarrollado cumple la mitad de los objetivos planteados, proporcionando una base sólida sobre la cual se puede integrar la funcionalidad faltante.